A 30-Round Asset Call Market as a Pure EDSL State Machine

Replication note

October 2026

Abstract

This note documents a 30-period, 12-trader asset market implemented as an EDSL `Machine`. The market mechanics—order admission, price-time priority, uniform-price clearing, account transfers, interest, dividends, and terminal redemption—are expressed in the Machine's serializable expression language. The machine declares `algorithms=()`, so the market does not delegate these operations to a custom Python algorithm or to EDSL's native call-market factory. One completed `gpt-4o` pilot produced 360 trader decisions and settled all 30 periods. Eight units traded in five periods, at prices from $15.00 to $15.14. This is a documented pilot, not a statistical replication: one market is insufficient to estimate the paper's market-level outcomes.

Contents

1 The experiment **2**

2 Machine definition **2**
- 2.1 State and transitions . . . 2
- 2.2 Allocation and uniform price . . . 3
- 2.3 Fair-value benchmark . . . 3

3 How one market run proceeds **3**

4 Implementation in EDSL **4**

5 Pilot results **7**
- 5.1 Completion and activity . . . 7
- 5.2 Trading styles and allocation . . . 8
- 5.3 Forecasts . . . 8

6 What the pilot says about the implementation **9**

7 Interpretation and next replication steps **9**

A Workflow source: launching and running the experiment **9**

B Run environment and operational notes **11**

C Reproducibility files **11**

1 The experiment

The experiment ports a baseline multi-period asset market into a dedicated study directory. It contains the Python authoring code, prepared workflow, completed run database, flat analysis outputs, and this report. The run analyzed here is `pilot_02`, seed 153101, using `gpt-4o` through EDSL's OpenAI service. The run completed 360 of 360 trader interviews, 30 of 30 clearings, and 30 of 30 settlements.

There are 12 traders, each starting with $100 and four shares, for 48 shares in total. In each of 30 periods, every trader submits one sealed buy, sell, or hold decision. A buy order's requested units are limited by available cash at the submitted limit price; a sell order is limited by current inventory. There is no borrowing or short selling. Each period, every share pays a common dividend, either $0.40 or $1.00 with equal probability. Cash earns 5 percent interest per period. After period 30's interest and dividend, each remaining share is redeemed for $14.

Four traders receive a value-buyer instruction, four a value-seller instruction, and four a flexible-value instruction. The instructions describe trading tendencies, not hard-coded trading policies. Each model interview sees the trader's current account and history plus the public market tape, and returns four price forecasts and one order decision. Price limits are requested in integer cents to avoid floating point conversion inside the market logic.

2 Machine definition

2.1 State and transitions

We can describe the machine as

$$M = (X, C, A, V, \delta, \mathcal{K}),$$

where X is the typed mutable state, C is the constant data embedded when the Machine is authored, A is the set of named commands, V is the agent-facing view, δ is the command transition semantics, and $\mathcal{K}$ is the set of registered runtime algorithms. Here $\mathcal{K} = \varnothing$. EDSL's standard interpreter evaluates the serializable expressions in δ; the absence of callbacks does not mean the Machine runs without an interpreter.

The mutable state is

$$X_t = (\text{accounts}_t, \text{orders}_t, \text{clearing}_t, \text{orderLog}_t, \text{tape}_t, p_t, \ell_t, f_t),$$

with per-trader cash in cents, share inventory, and private history inside `accounts`; current-period submitted orders in `orders`; the pending allocation in `clearing`; public history in `order_log` and `tape`; current period p_t; last transaction price ℓ_t; and terminal flag f_t. The view exposes the public tape and the current trader's own account, but does not expose other traders' private account histories. The initial state has 12 identical accounts, 10,000 cents and four shares per account, period 1, reference price 1,400 cents, empty orders and histories, and $f_1 =$ false.

There are three transitions, all represented as `Command` expressions:

1. **`submit_order(i,t,d)`** requires an unfinished market, the current period, a known trader, and no order already submitted by that trader in this period. It validates hold versus buy/sell fields, caps buy quantity at $\min(q_{requested}, \lfloor C_i/L_i \rfloor, 48)$, and caps sell quantity at $\min(q_{requested}, S_i)$. It stores the accepted decision. The submission does not directly mutate cash or shares.

2. **`clear_orders(t)`** requires exactly 12 submitted orders and no existing pending clearing. It expands each accepted order into unit bids or asks, sorts them by price and seeded priority, and computes the maximum crossing volume. It stores the resulting matched units, fills, and a single uniform transaction price in `clearing`.

3. `settle(t)` requires a pending clearing. It applies cash and share transfers, interest, dividends, histories, the public tape, and (in period 30) redemption. It clears the order book and pending allocation, advances the period, and sets `finished` after the final period.

2.2 Allocation and uniform price

Expand each admitted order of size q into q unit orders. Let the ranked bid limits be $b_1 \geq b_2 \geq \ldots$, and ask limits $a_1 \leq a_2 \leq \ldots$. Ties use a seeded period-specific trader priority; unit index makes multiple units from one trader deterministic. The matched quantity is

$$Q_t = \max(\{0\} \cup \{q \in \{1, \ldots, \min(n_b, n_a)\} : b_q \geq a_q\}),$$

with the empty match represented by $Q_t = 0$. If $Q_t > 0$, all matched units trade at

$$P_t = \text{round}_{\text{half-up, cent}}\left(\frac{b_{Q_t} + a_{Q_t}}{2}\right).$$

Thus the price is the rounded midpoint of the marginal matched bid and ask. The highest bids and lowest asks receive the fills. Unfilled orders expire at settlement. If no units cross, no transaction price is recorded and the last transaction price remains the public reference price.

For trader i, define x_{it} as filled purchases minus filled sales, and C_{it}^-, S_{it}^- as cash and inventory before the trade. At a trade price P_t measured in cents, the post-trade balances are

$$\widetilde{C}_{it} = C_{it}^- - x_{it}P_t, \qquad \widetilde{S}_{it} = S_{it}^- + x_{it}.$$

The period income and end balances are

$$I_{it} = \text{round}_{\text{half-up}}(0.05\widetilde{C}_{it}), \quad D_t \in \{40, 100\} \text{ cents}, \quad C_{it}^+ = \widetilde{C}_{it} + I_{it} + D_t\widetilde{S}_{it} + R_{it},$$

where $R_{it} = 1400\widetilde{S}_{it}$ in period 30 and zero otherwise. After redemption, the period-30 inventory is set to zero. All arithmetic for prices, cash, interest, dividends, and redemption is performed in integer cents; ratios explicitly specify floor or half-up rounding.

2.3 Fair-value benchmark

The expected dividend is \$0.70 per share per period, exactly five percent of the \$14 redemption value. Under risk-neutral discounting at the cash interest rate, the value at the beginning of any period with n dividend payments remaining is

$$V_n = \sum_{k=1}^{n} \frac{0.70}{(1.05)^k} + \frac{14}{(1.05)^n} = 14.$$

This \$14 benchmark is a useful reference for interpreting prices, conditional on the stated common dividend process, risk-neutral valuation, and the 5 percent cash rate. It is not a claim that all model traders share those assumptions.

3 How one market run proceeds

The workflow uses one session-scoped `SharedStateMap` and one instance of the Machine for the full market. Each period has three ordered workflow steps:

1. **Orders:** the 12 trader role members each read the same current shared state, but receive an account view personalized to their identity. Their responses are written with `submit_order`. Interviews can run concurrently, while each writes only its own map entry.

2. **Clearing:** after all trader interviews finish, a scripted exchange participant invokes `clear_orders`. The question text asks for no arithmetic; the actual allocation and price are evaluated by the Machine expression.
3. **Settlement:** the exchange step invokes `settle`. It applies the allocation and income and closes the period's book.

The next period depends on completion of settlement. This sequence repeats 30 times, giving 90 workflow steps, 360 trader interviews, 30 clearing transitions, and 30 settlement transitions. The persisted state log contains 360 reads and 420 writes: one decision write for each trader interview, one clearing write and one settlement write per period.

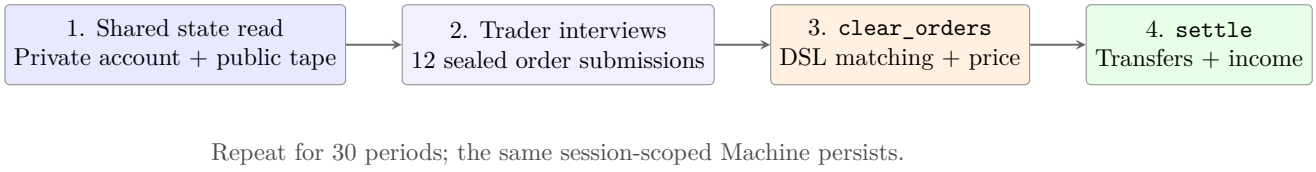


Figure 1: One period of the workflow. The order interviews can run in parallel; clearing and settlement wait on their respective predecessor steps.

4 Implementation in EDSL

The expression builders assemble serializable tree nodes. Python is used to author the tree and to materialize the seeded exogenous schedules. The schedule is part of the Machine constants, so no Python random draw or market callback is needed while executing a transition. The builder calls `validate()` before returning the Machine.

The source's unit expansion and crossing scan make the matching rule explicit:

Listing 1: Source excerpt: expanding and ranking unit orders, then scanning crossing bids and asks.

```
def _expanded_units(side: str):
    ordered = reduce(
        "sort_records",
        filter_items(
            field("orders").values(),
            item="order",
            predicate=(local("order").get("side") == side)
            & (local("order").get("accepted_quantity") > 0),
        ),
        fields=("limit_cents", "priority"),
        descending=(side == "buy", False),
    )
    return fold(
        ordered,
        [],
        item="order",
        accumulator="units",
        body=let(
            "expanded",
            iterate(
                record(index=0, result=local("units")),
                state="unit_cursor",
                until=(
                    local("unit_cursor").get("index")
                    >= local("order").get("accepted_quantity")
                ),
                step=record(
                    index=local("unit_cursor").get("index") + 1,
                    result=local("unit_cursor").get("result").appended(
                        record(
```

```
                            trader=local("order").get("trader"),
                            seat=local("order").get("seat"),
                            priority=local("order").get("priority"),
                            limit_cents=local("order").get("limit_cents"),
                            unit_index=local("unit_cursor").get("index"),
                        )
                    ),
                ),
                max_steps=constant("maximum_buy_quantity"),
            ),
            local("expanded").get("result"),
        ),
    )


def _ranked_units(side: str):
    return reduce(
        "sort_records",
        _expanded_units(side),
        fields=("limit_cents", "priority", "unit_index"),
        descending=(side == "buy", False, False),
    )


def _volume_scan():
    bids, asks = local("ranked_bids"), local("ranked_asks")
    return iterate(
        record(index=0, volume=0),
        state="scan",
        until=(
            (local("scan").get("index") >= bids.length())
            | (local("scan").get("index") >= asks.length())
            | (
                bids.at(local("scan").get("index")).get("limit_cents")
                < asks.at(local("scan").get("index")).get("limit_cents")
            )
        ),
        step=record(
            index=local("scan").get("index") + 1,
            volume=local("scan").get("volume") + 1,
        ),
        max_steps=constant("total_shares"),
    )
```

The actual Machine declaration below defines typed state fields, command preconditions, and state effects. These are Python builder calls; the market logic within command effects is still encoded as EDSL expressions, not an imperative callback body.

Listing 2: Source excerpt: seeded constants, initial accounts, and typed state.

```
def build_machine(seed: int = 153101) -> Machine:
    seats = {trader: seat for seat, trader in enumerate(TRADERS)}
    constants = {
        "seed": seed,
        "periods": PERIODS,
        "traders": list(TRADERS),
        "trader_count": len(TRADERS),
        "total_shares": len(TRADERS) * INITIAL_SHARES,
        "maximum_buy_quantity": MAXIMUM_BUY_QUANTITY,
        "initial_reference_cents": INITIAL_REFERENCE_CENTS,
        "redemption_cents": REDEMPTION_CENTS,
        "interest_numerator": INTEREST_NUMERATOR,
        "interest_denominator": INTEREST_DENOMINATOR,
        "dividend_schedule": _dividend_schedule(seed),
        "priority_schedule": _priority_schedule(seed),
    }
    machine = Machine(
        name="PureEDSLCallMarket30Round",
        constants=constants,
        fields={
```

```
            "accounts": state_field(
                StateType.map(),
                {
                    trader: {
                        "seat": seats[trader],
                        "cash_cents": INITIAL_CASH_CENTS,
                        "shares": INITIAL_SHARES,
                        "history": [],
                        "redeemed_shares": None,
                    }
                    for trader in TRADERS
                },
            ),
            "orders": state_field(StateType.map(), {}),
            "clearing": state_field(StateType.optional(StateType.map()), None),
            "order_log": state_field(StateType.sequence(), []),
            "tape": state_field(StateType.sequence(), []),
            "period": state_field(StateType.integer(minimum=1), 1),
            "last_price_cents": state_field(
                StateType.integer(minimum=0), INITIAL_REFERENCE_CENTS
            ),
            "finished": state_field(StateType.boolean(), False),
        },
        commands={
```

Listing 3: Source excerpt: the three Machine commands, agent view, empty algorithm registry, and validation.

```
        commands={
            "submit_order": Command(
                inputs={
                    "trader": StateType.text(),
                    "period": StateType.integer(minimum=1),
                    "decision": StateType.map(),
                },
                require=(
                    (~field("finished"))
                    & (arg("period") == field("period"))
                    & field("accounts").contains(arg("trader"))
                    & (~field("orders").contains(arg("trader")))
                ),
                effects=(
                    put(
                        "orders",
                        arg("trader"),
                        let("trader", arg("trader"), let("decision", arg("decision"), _admitted_order())),
                        once=True,
                    ),
                ),
            ),
            "clear_orders": Command(
                inputs={"period": StateType.integer(minimum=1)},
                require=(
                    (~field("finished"))
                    & (arg("period") == field("period"))
                    & (field("orders").length() == constant("trader_count"))
                    & (field("clearing") == None)
                ),
                effects=(assign("clearing", _clearing_record()),),
            ),
            "settle": Command(
                inputs={"period": StateType.integer(minimum=1)},
                require=(
                    (~field("finished"))
                    & (arg("period") == field("period"))
                    & (field("clearing") != None)
                ),
                effects=(
                    assign("accounts", _updated_accounts()),
```

```
                    assign("order_log", _order_log_after_clear()),
                    assign("tape", _tape_after_clear()),
                    assign(
                        "last_price_cents",
                        choose(
                            field("clearing").get("volume") > 0,
                            field("clearing").get("price_cents"),
                            field("last_price_cents"),
                        ),
                    ),
                    assign("orders", {}),
                    assign("clearing", None),
                    assign("finished", field("period") == constant("periods")),
                    assign("period", field("period") + 1),
                ),
            ),
        },
        view={
            "period": field("period"),
            "tape": field("tape"),
            "clearing_pending": field("clearing"),
            "last_price": field("last_price_cents") / 100,
            "your_account": field("accounts").get(current("name")),
            "finished": field("finished"),
        },
        complete_when=field("finished"),
        algorithms=(),
    )
    machine.validate()
```

5 Pilot results

5.1 Completion and activity

All 360 interviews were completed and all 30 periods settled. The Machine finished in period 31 with no pending orders, zero remaining shares, and 48 redeemed shares. The run logged 360 shared-state reads and 420 writes, exactly matching the workflow's planned decision and market transition counts.

There were 213 buy decisions, 89 sell decisions, and 58 holds. Those counts are choices, not executions. Only eight units traded, across five periods; 25 periods had no transaction. The market's active trading interval was periods 10 through 14. The price rose in each of these five periods, from \$15.00 to \$15.14; afterward the last transaction price was carried forward as the tape reference, but it was not a new market quote.

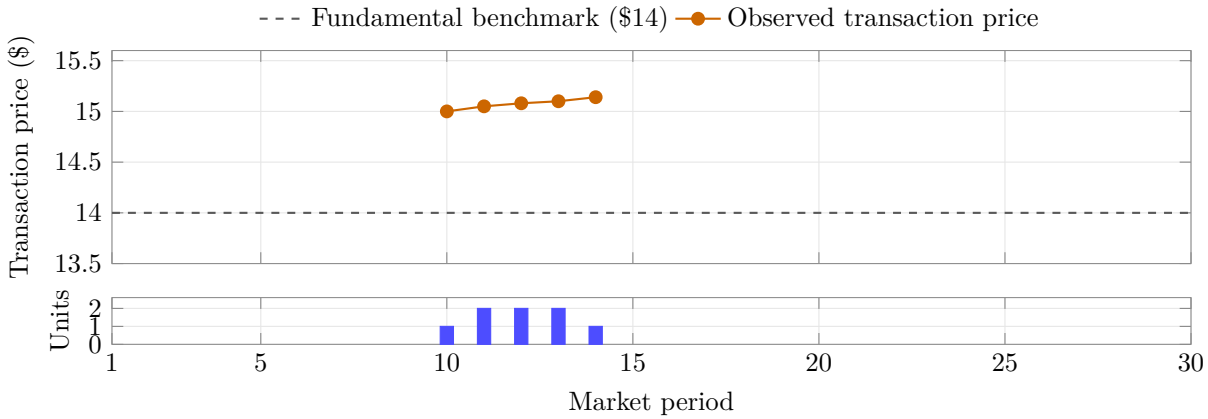


Figure 2: The five transaction observations and traded quantity. Periods without a marker had no transaction; the tape carries the last transaction price as a reference, not as a new clearing price.

Period	Transaction price	Units	Difference from \$14
10	\$15.00	1	7.1%
11	\$15.05	2	7.5%
12	\$15.08	2	7.7%
13	\$15.10	2	7.9%
14	\$15.14	1	8.1%
All trades	Mean \$15.074	8	7.7% above benchmark

Table 1: Observed transaction prices. Each row is one period's uniform price; the total quantity matched was eight units.

5.2 Trading styles and allocation

The model followed the broad direction of its assigned styles, although these were instructions rather than binding algorithms. The four value buyers made 117 buys and three holds. Value sellers made 87 sells and 33 holds. Flexible value traders made 96 buys, 22 holds, and two sells. In this run, the flexible role leaned strongly toward buying. The 8 purchased units went to flexible traders; 8 units were sold by two value sellers. Every other share remained with its initial holder and was redeemed at the end.

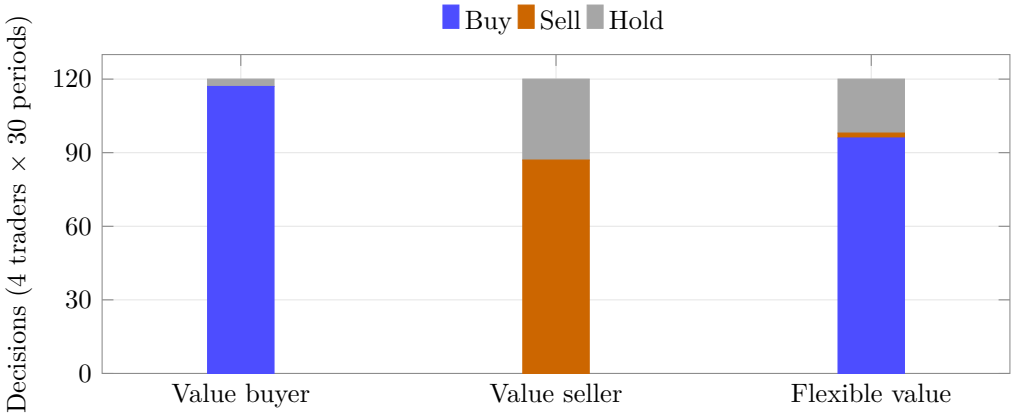


Figure 3: Actions by assigned style, aggregated across each role's four traders and all 30 periods.

Total trader cash after redemption was \$8,150.35. Final balances ranged from \$661.18 to \$690.67. All traders started with \$100 and four shares, whose combined terminal redemption value was \$672 per trader before interest, dividends, and trading. Two value sellers sold all four of their shares before redemption; flexible traders bought those eight shares. The spread of final cash thus reflects dividend and interest receipts, plus the timing and price of those transfers.

5.3 Forecasts

Mean reported forecasts were \$14.903 for the current-period price, \$15.005 two periods ahead, \$15.135 five periods ahead, and \$15.311 ten periods ahead. Mean absolute errors against realized transaction prices were \$0.184, \$0.566, \$0.710, and \$0.503, respectively. These errors are computed only when a transaction exists at the forecast target: the sample sizes are 60 for horizons 0, 2, and 5, and 48 for horizon 10. They are not errors over all forecasts or over unobserved no-trade periods. Because prices were observed in only five periods, these diagnostics should be read as sparse pilot summaries.

6 What the pilot says about the implementation

The DSL carried the market mechanics. The Machine declares three commands, typed state and inputs, preconditions, and effects. The market's allocation, price, settlement, interest, dividend, and redemption are all composed from built-in expressions. In the experiment's metadata and serialized definition, the algorithm list is empty. This is direct evidence that this version is not a wrapper around the native `call_market` factory and does not subclass that market's transition implementation.

Machine serialization is useful, with a boundary. The definition serializes to JSON and contains its seeded schedules and expression trees. It can therefore be inspected, stored, validated, and replayed by an EDSL runtime. The runtime remains necessary to interpret the expression language and enforce command preconditions; JSON alone is a data representation, not an executable interpreter.

The larger per-operation budget is necessary for full-history state. The default one-million-step interpreter ceiling was exhausted by a late-round transition in an earlier full-horizon attempt. The experiment sets the ceiling to ten million steps. The Machine deliberately retains every order and each trader's period history for inspection, so transition work grows over time. The deterministic 30-period mechanics smoke and this live run both reached terminal completion with the larger limit.

Model inference dominates observed runtime. Log timing for this pilot showed roughly 38–40 seconds for a batch of 12 trader interviews, while the clearing transition averaged about 1.47 seconds and settlement about 1.85 seconds. The two DSL transitions therefore took about 3.3 seconds per round, small beside the model calls. The shared-state version adds measurable expression evaluation and state-log work, but the observed end-to-end delay is primarily the language-model inference.

7 Interpretation and next replication steps

This run establishes that the full-period mechanics can be encoded in the Machine expression language and executed through the shared-state workflow, including account constraints and terminal cash-out. It does not establish that the market reproduces the paper's behavioral or price outcomes. The market traded only eight units, and prices were observed only in five periods. The 15-cent range in those five prints is descriptive; with one seeded market it cannot be treated as an estimate of typical price convergence, mispricing, or volume.

A paper-level comparison should run multiple independent markets and compare the same market-level statistics reported in the source paper, with uncertainty at the market level. It should also inspect order submission, fill rates, trading volume, price paths, and trader earnings by strategy. Forecast metrics should continue to distinguish transaction-observed targets from periods with no transaction. The saved run is suitable as a mechanics and workflow pilot; its eight-unit tape is too sparse to support stronger economic conclusions.

A Workflow source: launching and running the experiment

The experiment creates a single Machine-backed shared state, binds the trader role to a language model and the exchange role to scripted answers, then chains orders, clearing, and settlement for each period. The clearing and settlement answers are merely workflow responses; all effects come from the attached Machine commands.

Listing 4: Source excerpt: agents, session-scoped Machine state, and execution-plan bindings.

```
def build(*, seed: int = 153101, model: str = "gpt-4o", service: str = "openai"):
```

```
    strategies = ["value_buyer"] * 4 + ["value_seller"] * 4 + ["flexible_value"] * 4
    agents = [
        Agent(
            name=name,
            traits={"role": "trader", "strategy": strategies[seat], "seat": seat},
            instruction=SYSTEM + "\n\nTRADING STYLE\n" + STYLE[strategies[seat]],
        )
        for seat, name in enumerate(TRADERS)
    ]
    agents.append(Agent(name="exchange", traits={"role": "exchange"}))

    states = SharedStateMap(
        SharedState(market=build_machine(seed)),
        state_id=f"pure-edsl-call-market-seed-{seed}",
    )
    market = states.by("session").market
    workflow = Workflow(
        "30-period pure EDSL call market",
        metadata={
            "seed": seed,
            "periods": PERIODS,
            "traders": len(TRADERS),
            "machine": "PureEDSLCallMarket30Round",
            "custom_market_algorithms": [],
            "strategy_composition": {key: 4 for key in STYLE},
            "price_input": "integer cents",
            "created_at": datetime.now(timezone.utc).isoformat(),
        },
    )
    plan = ExecutionPlan().bind(
        role("trader"),
        llm(model=model, service=service, parameters={"temperature": 0.2}),
    ).bind(
        role("exchange"),
        scripted(answers={"clear": "matched", "settlement": "settled"}),
    )
```

Listing 5: Source excerpt: chained order, clearing, and settlement steps, and WorkflowExperiment construction.

```
    previous = None
    for period in range(1, PERIODS + 1):
        decision = _decision_question(period)
        orders = workflow.step(
            f"orders-{period}",
            Survey([decision]),
            assigned_to=role("trader"),
            after=previous,
            visible_to=role("exchange"),
            reads=(market.read(),),
            writes=(
                market.submit_order(
                    trader=current.agent.name,
                    period=period,
                    decision=decision.answer,
                ),
            ),
            metadata={"period": period, "phase": "sealed orders"},
        )
        cleared = workflow.step(
            f"clear-{period}",
            Survey(
                [
                    QuestionFreeText(
                        question_name="clear",
                        question_text="Compute the crossing allocation from the machine's EDSL expressions.",
                    )
                ]
            ),
```

```
            assigned_to=role("exchange"),
            after=orders,
            writes=(market.clear_orders(period=period),),
            metadata={"period": period, "phase": "machine price priority and matching"},
        )
        settled = workflow.step(
            f"settle-{period}",
            Survey(
                [
                    QuestionFreeText(
                        question_name="settlement",
                        question_text="Apply the EDSL clearing result, account transfers, income, and final
                            redemption.",
                    )
                ]
            ),
            assigned_to=role("exchange"),
            after=cleared,
            writes=(market.settle(period=period),),
            metadata={"period": period, "phase": "machine settlement"},
        )
        previous = settled

    return WorkflowExperiment(
        workflow.compile(),
        [states],
        agents,
        plan,
        metadata={
            "seed": seed,
            "economic_horizon": PERIODS,
            "strategy_by_trader": {
                agent.name: agent.traits.get("strategy")
                for agent in agents
                if agent.traits["role"] == "trader"
            },
            "machine_json_bytes": len(build_machine(seed).to_json()),
            "custom_market_algorithms": [],
        },
        runtime=MACHINE_RUNTIME,
    )
```

The command used to launch the completed pilot was:

```
python3 edsl_replication/studies/asset_market_pure_edsl_30r/experiment.py \\
  --output edsl_replication/studies/asset_market_pure_edsl_30r/pilot_02
```

The run artifacts include the serialized specification and experiment, workflow and state databases, completion record, model-call log, and the derived CSV and JSON summaries.

B Run environment and operational notes

The only run analyzed here is the successful `pilot_02` market. The prepared workflow and persisted database confirm 360 decisions and 30 complete periods. An earlier full-horizon mechanics attempt reached the default one-million-step expression ceiling late in the run; the 10-million-step configuration allowed the deterministic smoke and the model-backed market to complete. Initial local object-cache permission errors were retried and are not part of the economic results. They did not reduce the completed sample.

C Reproducibility files

The implementation and outputs are stored alongside this PDF:

- `machine_definition.py`: the Machine and serializable expression trees.
- `experiment.py`: agents, questions, shared state, workflow, and run entry point.
- `pilot_02/specification.json`: prepared serializable workflow and machine.
- `pilot_02/results_summary.json`: aggregate counts and descriptive statistics.
- `pilot_02/round_summary.csv`: period-level tape and income fields.
- `pilot_02/trader_summary.csv`: strategy, final balances, orders, and fills by trader.

The analysis script `analyze_run.py` regenerates the two CSV files and summary JSON from the saved state and workflow databases. For a new independent market, use a fresh output directory and a different seed. The deterministic mechanics-only smoke is available as `smoke_run.py`.